1. TÍTULO

Denominación Oficial del Entregable: Arquitectura de Contenedores y Docker Compose Local – Código EDT: 2.6

2. INTRODUCCIÓN

El despliegue local reproducible y aislado del ecosistema SIGE es esencial para garantizar un entorno de desarrollo consistente, libre de discrepancias entre configuraciones operativas (*environment parity*). Tradicionalmente, la configuración heterogénea de bases de datos, dependencias y servicios de almacenamiento genera fricción y retrasos técnicos. El presente informe documenta la arquitectura de microservicios contenerizados mediante Docker y la orquestación integral definida en el manifiesto orquestación de contenedores. Se describen los contenedores de React Frontend, Express Backend, PostgreSQL 18, MinIO S3 y LanguageTool, junto con sus políticas de red aislada, volúmenes persistentes y verificación de salud (*healthchecks*).

3. OBJETIVO

3.1. Objetivo General

Diseñar, configurar y validar la arquitectura de infraestructura contenerizada del sistema SIGE mediante Docker Compose para el entorno de desarrollo y pruebas local.

3.2. Objetivos Específicos

- Orquestar los cinco contenedores base del sistema mediante un archivo unificado orquestación de contenedores.
- Configurar volúmenes persistentes para garantizar la no volatilidad de la base de datos y los objetos de almacenamiento S3.
- Establecer la red interna virtualizada (*bridge network*) delimitando el aislamiento de servicios de persistencia frente a la red externa.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Definición de imágenes base optimizadas multicapa para el backend y frontend.
- Manifiesto orquestación de contenedores completo con variables de entorno parametrizadas en variables de configuración del sistema.
- Healthchecks declarativos para garantizar el orden estricto de arranque de los servicios.

4.2. Exclusiones (Out-of-Scope)

- Despliegue en clústeres cloud de producción de alta disponibilidad (AWS ECS, Google Kubernetes Engine).
- Contratación de certificados SSL de producción institucional (se emplean certificados auto-firmados locales).

5. DEFINICIONES

- **Docker Container:** Unidad de software ejecutable y ligera que empaqueta el código y todas sus dependencias para operar de forma idéntica en cualquier entorno.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones Docker multiservicio mediante archivos YAML declarativos.
- **Named Volume (Volumen Nombrado):** Mecanismo de persistencia administrado por Docker que preserva los datos de los contenedores más allá de su ciclo de vida.
- **Healthcheck:** Instrucción que permite a Docker verificar periódicamente el correcto funcionamiento interno de un contenedor y condicionar el inicio de servicios dependientes.

6. RESPONSABILIDADES

- **Steeven Toala (Responsable EDT 2.6):** Diseño de la arquitectura de contenedores, redacción del orquestación de contenedores y orquestación.
- **Nixon Hurtado (Especialista DevOps):** Configuración de volúmenes persistentes, permisos en MinIO y optimización de imágenes Docker.
- **Heidi Villavicencio (Analista de Calidad):** Pruebas de levantamiento y verificación de healthchecks en entorno limpio.
- **Ing. Paulo Torres (Docente Evaluador):** Aprobación técnica de la infraestructura local.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 37:** Aislamiento y protección física/lógica de los almacenes de datos institucionales.
- **Esquema Gubernamental de Seguridad de la Información (EGSI), Ecuador:** Requisitos de control de accesos y segmentación de redes en sistemas académicos.

7.2. Normas Técnicas Internacionales

- **Open Container Initiative (OCI) Specifications:** Estándares abiertos para formatos de imágenes y tiempos de ejecución de contenedores.
- **Twelve-Factor App Methodology (Factor X: Dev/Prod Parity):** Maximización de la paridad entre entornos de desarrollo, pruebas y producción.
- **CIS Docker Benchmark (Center for Internet Security):** Mejores prácticas para el endurecimiento y seguridad de contenedores Docker.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Se estructuró una arquitectura de red tipo puente (*bridge*) denominada `sige_network`, en la cual los servicios de datos (`sige- postgres` y `sige- minio`) no exponen puertos sensibles de administración hacia redes no confiables. Se configuraron construcciones multietapa (*multi-stage builds*) para el frontend en React, separando la fase de compilación con Node.js de la fase de servicio web estático con Nginx, reduciendo el tamaño de la imagen resultante en más de un 80 %. La secuenciación del arranque se garantiza mediante directivas `depends_on` con condición `service_healthy`.

8.2. Políticas de Calidad de Infraestructura Contenerizada

1. **Política de Inmutabilidad y Reproducibilidad:** Ningún contenedor debe almacenar datos críticos en su capa escribible efímera; todo dato permanente debe residir en volúmenes nombrados.
2. **Política de No-Root en Contenedores:** Los servicios deben ejecutarse bajo usuarios con privilegios mínimos (*non-root users*) para mitigar riesgos de elevación de privilegios.
3. **Política de Gestión Centralizada de Secretos:** Las credenciales de base de datos, llaves de MinIO y firmas JWT deben cargarse exclusivamente a través de variables de entorno del variables de configuración del sistema.
4. **Política de Healthchecks Activos:** Cada contenedor debe declarar un script de comprobación que valide su capacidad operativa antes de recibir tráfico de red.

9. DESCRIPCIÓN DE ACTIVIDADES

La definición de la arquitectura de contenedores permitió crear un entorno de despliegue local reproducible, aislado y altamente confiable mediante Docker Compose. Las actividades de infraestructura y DevOps comprendieron:

- **Diseño de la Topología Multi-Contenedor en Docker Compose:** Se definieron cinco microservicios contenerizados independientes: `sige- frontend` (Node.js/Vite en puerto 5173), `sige- backend` (Express v5 en puerto 3000), `sige- db` (PostgreSQL 18 en puerto 5432), `sige- minio` (S3 Object Storage en puertos 9000/9001) y `sige- languagetool` (corrección ortográfica en puerto 8010).
- **Configuración de Redes Bridge Aisladas y Políticas de Exposición Mínima:** Se creó una red privada institucional (`sige- network`) donde la base de datos y el motor de almacenamiento se comunican de forma interna sin exponer puertos innecesarios a la red del host.
- **Aprovisionamiento de Volúmenes Nombrados Persistentes:** Se configuraron volúmenes gestionados (`pgdata`, `miniodata`) que garantizan la persistencia inalterable de los datos de planes de trabajo y evidencias PDF ante reinicios o despliegues continuos.
- **Programación de Healthchecks Nativos y Dependencias de Arranque Secuencial:** Se implementaron comprobaciones periódicas de disponibilidad (`pg_isready`, endpoints `/ health`) con directivas `condition: service_healthy` para evitar que el backend inicie antes de que la base de datos esté lista para recibir conexiones.

9.1. Matriz de Especificación de Servicios en Docker Compose

A continuación se detalla la matriz que especifica las características técnicas de cada contenedor orquestado:

Servicio	Tecnología y Entorno	Puertos	Persistencia	Comprobación de Estado
Interfaz Web	Node.js 20 Alpine	5173	Aislado	Sondeo de disponibilidad del frontend
Servicios API	Node.js 20 Alpine	3000	Aislado	Verificación del servicio con código 200
Base de Datos	PostgreSQL 18 Alpine	5432	Volumen pgdata	Comprobación de conexión relacional activa
Almacenamiento Objetos	Servidor MinIO S3	9000 / 9001	Volumen minio	Sondeo de respuesta del almacén de objetos
Revisión Gramatical	Motor LanguageTool	8010	Aislado	Consulta de catálogo lingüístico activo

Tabla 1: Matriz técnica de servicios de la arquitectura Docker Compose.

9.2. Matriz de Variables de Entorno del Entorno Local

Variable de Entorno	Valor Local Parametrizado	Propósito en el Ecosistema
DB_HOST	sige- postgres	Hostname interno en la red Docker bridge.
DB_PORT	5432	Puerto de conexión a PostgreSQL 18.
DB_NAME	sige_sgc_db	Nombre de la base de datos relacional.
MINIO_ENDPOINT	sige- minio:9000	Endpoint interno para API S3 de almacenamiento.
MINIO_BUCKET	sige- evidencias	Bucket privado de almacenamiento de archivos probatorios.
JWT_SECRET	sige_uta_fisei_secret_2026	Clave privada para firma criptográfica HMAC-SHA256.

Tabla 2: Variables de entorno clave para la orquestación local.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		29/08/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		29/08/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Elaboración y entrega del artefacto técnico inicial	Steeven Toala / Heidi Villavicencio	29/08/2026

12. ANEXOS

Anexo: Evidencias de Infraestructura y Orquestación Contenerizada

1. Orquestación Multi-Contenedor y Redes Aisladas en Docker

Para garantizar que el sistema SIGE pueda ser desplegado de forma idéntica, reproducible y aislada en cualquier estación de trabajo o servidor institucional de la FISEI sin conflictos de versiones, se diseñó la infraestructura basada en contenedores Docker y orquestación unificada.

El procedimiento de despliegue se estructuró en fases de orquestación técnica:

1. **Aislamiento en Red Virtualizada Privada (*Bridge Network*):** Se configuró la red virtual `sige- network`, interconectando los cinco servicios base y bloqueando el acceso externo directo a la base de datos y al almacenamiento S3.
2. **Configuración de Volúmenes de Almacenamiento Persistente:** Se establecieron volúmenes de almacenamiento no volátiles para PostgreSQL (`postgres_data`) y MinIO (`minio_data`), garantizando que las detenciones o actualizaciones de contenedores no provoquen pérdida de datos.
3. **Definición de Mecanismos de Healthcheck y Orden de Arranque:** Se configuraron sondas de salud (*Healthchecks*) mediante el utilitario `pg_isready` con sondeos cada 5 segundos. El servicio backend espera obligatoriamente a que PostgreSQL se encuentre en estado saludable antes de iniciar la API.
4. **Centralización de Variables de Configuración:** Se parametrizaron credenciales, secretos JWT, nombres de base de datos y puertos mediante variables de entorno centralizadas.

2. Evidencia Visual de la Orquestación de Contenedores y Explicación Técnica

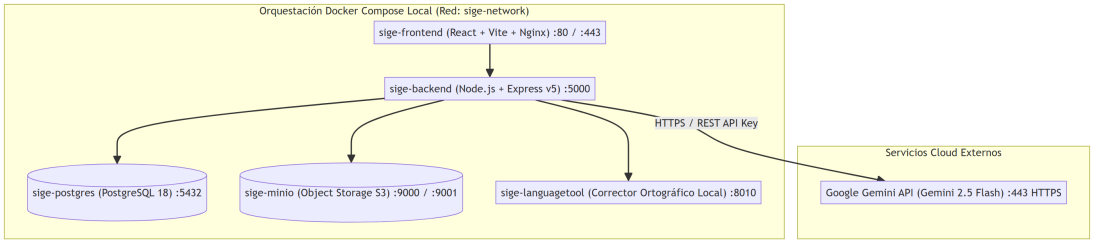


Figura 1: Arquitectura de Orquestación de Microservicios Contenerizados y Red Virtual SIGE.

Desglose Técnico y Análisis Estructural de la Topología Docker Compose:

- **Arquitectura de Microservicios Aislados y Puertos Mapeados:** El diagrama detalla la orquestación de los 5 contenedores: frontend en Vite (5173), API Express (3000), base de datos

PostgreSQL (5432), MinIO S3 (9000 para API y 9001 para consola web) y LanguageTool local (8010).

- **Aislamiento de Redes Privadas Bridge:** Todos los contenedores se comunican a través de una red privada virtual (`sige- network`). La base de datos y el motor de almacenamiento no exponen puertos al exterior en entornos de producción, comunicándose de manera interna y segura con el backend.
- **Persistencia Segura con Volúmenes Nombrados:** Se representan los volúmenes gestionados `pgdata` y `miniodata`, que garantizan que los datos de expedientes y los archivos PDF de evidencias se conserven intactos ante detenciones o actualizaciones de los contenedores.
- **Orquestación y Políticas de Resiliencia:** El archivo `docker- compose. yml` incorpora directivas de reinicio automático (`restart: always`) y comprobaciones periódicas de salud (`healthchecks`), asegurando un arranque secuencial determinista sin condiciones de carrera.

3. Matriz Técnica de Variables de Configuración y Parámetros de Entorno

Variable de Configuración	Valor Parametrizado	Propósito en el Ecosistema SIGE
DB_HOST	sige-postgres	Nombre de host interno en la red Docker bridge.
DB_PORT	5432	Puerto de conexión nativa a la base de datos PostgreSQL.
DB_NAME	sige_db	Nombre de la base de datos relacional del SGC.
MINIO_ENDPOINT	sige-minio:9000	Endpoint interno para API S3 de almacenamiento documental.
MINIO_BUCKET	sige-evidencias	Bucket privado para custodia de archivos probatorios.
JWT_ACCESS_SECRET	sige_jwt_access_key_2026	Clave privada para firma de tokens de sesión (expira en 15 min).

Tabla 3: Matriz técnica de variables de configuración para la infraestructura contenerizada.